

# Benefits of Moving from Java 8 to Java 21

---

This document explains, in simple language, what we can do with **Java 21** that was hard or impossible in **Java 8**, using examples from our **banking system**.

Each section follows the same structure:

- **The problem in Java 8** — what was painful
  - **The Java 21 solution** — how it gets easier
  - **Banking example** — code from our domain
  - **Why the bank should care** — business value
- 

## 1. Virtual Threads — Handle Many More Customers at the Same Time

---

### The problem in Java 8

When the bank processes payments, transfers, or fraud checks, each request usually runs on its own **thread**. A thread in Java 8 is heavy: it uses about **1 MB of memory** and the operating system can only manage a few thousand of them at once.

So if you want to handle **10 000 customers** transferring money at the same time, you cannot just create 10 000 threads. You must use a **thread pool** (for example, 200 threads) and queue everything else. When a thread is waiting for the database to respond, it just sits there, blocked, doing nothing — but still taking 1 MB of memory.

This is why Java 8 banks often have to add **more servers** during Black Friday or salary day: each server can only do so much.

### The Java 21 solution

Java 21 brings **Virtual Threads**. They look and behave exactly like normal threads, but they are **almost free**: about 1 KB instead of 1 MB. You can create **hundreds of thousands** of them on a single server.

When a virtual thread is waiting (for the database, for an external API, for the network), it gets out of the way and lets others work. When the answer comes back, it resumes.

## Banking example

### Java 8 — thread pool, must be tuned carefully:

```
ExecutorService executor = Executors.newFixedThreadPool(200);

for (TransferRequest req : todaysSalaryPayments) {
    executor.submit(() -> transactionService.transfer(
        req.fromAccount(), req.toAccount(), req.amount(), "Salary"
    ));
}
```

If 50 000 salary payments arrive at 9:00 AM, 49 800 of them sit in a queue.

### Java 21 — one virtual thread per payment:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (TransferRequest req : todaysSalaryPayments) {
        executor.submit(() -> transactionService.transfer(
            req.fromAccount(), req.toAccount(), req.amount(), "Salary"
        ));
    }
}
```

All 50 000 payments run at the same time. The same server handles them.

## Why the bank should care

- **Lower hardware cost.** The same machine can serve **10× to 100× more customers**.
- **Faster customer experience.** No more “please wait” during peak hours.
- **Simpler code.** Developers write straight, top-to-bottom logic instead of complicated callback chains.

---

## 2. Records — Less Code, Fewer Bugs

---

### The problem in Java 8

A bank works with many small objects: a `Transaction`, an `Address`, a `CustomerSummary`, a `Receipt`. In Java 8, every such class needs:

- A constructor
- A getter for each field
- `equals()` and `hashCode()`

- A `toString()` for logs
- Sometimes a `Builder`

For a class with 8 fields, this is easily **80 lines of repetitive code**. Every line is a place where a developer could make a typo, forget a field in `equals()`, or copy-paste a wrong getter.

## The Java 21 solution

A **Record** is a one-line declaration that gives you all of the above automatically. The compiler writes the boilerplate; the developer writes the meaning.

## Banking example

Java 8 — our `Transaction` is around 60 lines:

```
public class Transaction {
    private final String id;
    private final String fromAccountId;
    private final String toAccountId;
    private final BigDecimal amount;
    private final String currency;
    private final TransactionType type;

    public Transaction(String id, String from, String to,
                      BigDecimal amount, String currency, TransactionType type) {
        this.id = id;
        this.fromAccountId = from;
        // ...and so on for every field
    }

    public String getId() { return id; }
    public String getFromAccountId() { return fromAccountId; }
    // ...one getter per field

    @Override
    public boolean equals(Object o) { /* 15 lines */ }

    @Override
    public int hashCode() { /* 5 lines */ }

    @Override
    public String toString() { /* 5 lines */ }
}
```

Java 21 — the same thing, one line:

```
public record Transaction(  
    String id,  
    String fromAccountId,  
    String toAccountId,  
    BigDecimal amount,  
    String currency,  
    TransactionType type  
) {}
```

The compiler creates the constructor, accessors, `equals`, `hashCode`, and `toString` for you.

## Why the bank should care

- **Fewer bugs.** Less hand-written code means fewer mistakes in money-related objects.
- **Faster development.** A new “Receipt” or “StatementLine” object takes seconds to create.
- **Easier code review.** Reviewers see what is unique about a class instead of skimming through 60 lines of boilerplate.

---

## 3. Pattern Matching with Sealed Types – The Compiler Catches Missing Cases

---

### The problem in Java 8

Banking software is full of “decide what to do based on the type” logic. For example, displaying a transaction:

- `DEPOSIT` → “Money in”
- `WITHDRAWAL` → “Money out”
- `TRANSFER` → “Sent to another account”
- `FEE` → “Bank fee”

In Java 8 this is written with `if / else if` and `switch` statements. The big problem: **if a developer adds a new transaction type later (for example `CHARGEBACK`)**, nothing reminds them to update every `switch` in the codebase. The new type silently falls through `default`, and the bank shows wrong information to customers.

### The Java 21 solution

**Sealed types** let us tell the compiler: “These are the only possible kinds of Transaction.” Combined with **pattern-matching `switch`**, the compiler now **refuses to compile** if any case is missing.

## Banking example

### Java 8 — easy to forget a case:

```
String describe(Transaction t) {  
    if (t.getType() == TransactionType.DEPOSIT) {  
        return "Money received: " + t.getAmount();  
    } else if (t.getType() == TransactionType.WITHDRAWAL) {  
        return "Money withdrawn: " + t.getAmount();  
    } else if (t.getType() == TransactionType.TRANSFER) {  
        return "Transfer to " + t.getToAccountId();  
    }  
    // What about FEE? CHARGEBACK? Nothing reminds us.  
    return "Unknown";  
}
```

### Java 21 — the compiler protects us:

```
sealed interface Transaction  
    permits Deposit, Withdrawal, Transfer, Fee {}  
  
String describe(Transaction t) {  
    return switch (t) {  
        case Deposit d    -> "Money received: " + d.amount();  
        case Withdrawal w -> "Money withdrawn: " + w.amount();  
        case Transfer tr   -> "Transfer to " + tr.toAccountId();  
        case Fee f         -> "Bank fee: " + f.reason();  
    };  
}
```

If we add a new `Chargeback` type tomorrow, the code **will not compile** until we handle it everywhere. The bug is found before it reaches production.

## Why the bank should care

- **Safer changes.** Adding a new transaction type, account status, or product cannot accidentally break old screens.
  - **Clearer code.** Every possible case is listed in one place.
  - **Less manual testing.** The compiler does the “did I forget anything?” check for free.
-

## 4. Structured Concurrency — Cleaner Parallel Work

---

### The problem in Java 8

When a customer opens the **mobile app**, the bank often needs to load several things in parallel:

- Account balances
- Recent transactions
- Active cards
- Open loans

In Java 8 this is done with `CompletableFuture` and a lot of glue code. The hardest part is **failure**: if “load loans” crashes, what happens to the other three tasks? They keep running in the background, wasting resources and possibly returning data that the user will never see. If the user closes the screen, the same thing happens. Cleaning up properly is hard, and **leaks** happen.

### The Java 21 solution

**Structured Concurrency** treats parallel tasks like a small “team” that lives inside a clear scope. Rules:

- All tasks in the team finish (or fail) together.
- If one fails, the others are **automatically cancelled**.
- If the parent gives up (for example, the user navigates away), the whole team is cancelled.

### Banking example

**Java 8 — manual coordination, hard to get right:**

```
CompletableFuture<List<Account>>    accounts =
    CompletableFuture.supplyAsync(() -> accountService.findByCustomer(id));
CompletableFuture<List<Transaction>> txs =
    CompletableFuture.supplyAsync(() -> transactionService.history(id));
CompletableFuture<List<Card>>        cards =
    CompletableFuture.supplyAsync(() -> cardService.findByCustomer(id));
CompletableFuture<List<Loan>>        loans =
    CompletableFuture.supplyAsync(() -> loanService.findByCustomer(id));

CompletableFuture.allOf(accounts, txs, cards, loans).join();
// If loans fails, the other three keep running. We have to remember to cancel.
```

**Java 21 — clear, automatic, leak-proof:**

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {

    var accounts = scope.fork(() -> accountService.findByCustomer(id));
    var txs       = scope.fork(() -> transactionService.history(id));
    var cards     = scope.fork(() -> cardService.findByCustomer(id));
    var loans     = scope.fork(() -> loanService.findByCustomer(id));

    scope.join();           // wait for all
    scope.throwIfFailed();  // if any failed, the rest are already cancelled

    return new CustomerDashboard(
        accounts.get(), txs.get(), cards.get(), loans.get()
    );
}
```

If `loans` fails, the other three tasks are **immediately stopped**. No leaks, no half-results.

## Why the bank should care

- **Lower server load.** No “ghost” tasks doing work that nobody will use.
- **Better user experience.** The dashboard appears faster because all four queries run in parallel.
- **Safer code.** Cancellation and cleanup are handled by the language, not by the developer.

---

## 5. Text Blocks — Cleaner SMS, Receipts, and Audit Logs

---

### The problem in Java 8

A bank sends a lot of formatted text: **SMS notifications, email confirmations, printed receipts, audit log entries, SQL queries**. In Java 8, multi-line strings have to be glued together with `+` and `\n`. The result is hard to read, easy to break, and the formatting in code does not look like the formatting the customer will see.

### The Java 21 solution

**Text Blocks** let you write multi-line strings exactly the way they should look, between `"""` markers. What you see in the code is what the customer gets.

### Banking example

**Java 8 — a printed receipt:**

```
String receipt =
    "=====\n" +
    "      BANK RECEIPT\n" +
    "=====\n" +
    "Account:  " + account.getNumber() + "\n" +
    "Operation: " + type + "\n" +
    "Amount:   " + amount + " " + currency + "\n" +
    "Balance:  " + balance + "\n" +
    "Date:     " + date + "\n" +
    "Reference: " + ref + "\n" +
    "=====\\n";
```

### Java 21 — the same receipt, readable:

```
String receipt = ""
    =====
    BANK RECEIPT
    =====
    Account:   %s
    Operation: %s
    Amount:    %.2f %s
    Balance:   %.2f
    Date:      %s
    Reference: %s
    =====
    ""formatted(account.getNumber(), type, amount, currency,
                balance, date, ref);
```

### Why the bank should care

- **Fewer formatting bugs** in customer-facing text (a missing `\n` in an SMS looks unprofessional).
- **Easier to update** templates when marketing or compliance asks for a wording change.
- **Cleaner SQL** for reports — long queries become readable instead of a `+`-soup.

## 6. Modern HTTP Client — Easy Integration with Other Services

### The problem in Java 8

A modern bank rarely lives alone. It needs to talk to:

- **Central bank** for currency exchange rates

- **Fraud detection** services
- **SMS / email gateways**
- **Credit bureaus** (KYC, credit score)
- **Payment networks** (SWIFT, SEPA, card networks)

In Java 8, the built-in HTTP tool ( `URLConnection` ) is **clumsy**: dozens of lines for one request, no support for modern protocols (HTTP/2), no easy `async` . Most teams add an external library (Apache HttpClient, OkHttp), which means more dependencies to manage and update.

## The Java 21 solution

Java 21 includes a **modern, built-in HttpClient** : short syntax, supports HTTP/2 out of the box, and works perfectly together with virtual threads (see Section 1) — so even thousands of parallel external calls stay cheap.

## Banking example

### Java 8 — getting today's USD/EUR rate:

```
URL url = new URL("https://api.centralbank.example/rates?base=USD&target=EUR");
URLConnection conn = (URLConnection) url.openConnection();
conn.setRequestMethod("GET");
conn.setConnectTimeout(5000);
conn.setReadTimeout(5000);

try (BufferedReader reader = new BufferedReader(
    new InputStreamReader(conn.getInputStream()))) {
    StringBuilder sb = new StringBuilder();
    String line;
    while ((line = reader.readLine()) != null) sb.append(line);
    String json = sb.toString();
    // ...then parse JSON manually or with another library
}
```

### Java 21 — the same call:

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.centralbank.example/rates?base=USD&target=EUR"))
    .timeout(Duration.ofSeconds(5))
    .build();

String json = client.send(request, BodyHandlers.ofString()).body();
```

Combine this with **records** (Section 2) and parsing the response is just a few more lines.

## Why the bank should care

- **Faster integrations.** Connecting to a new fraud or KYC partner takes hours instead of days.
- **Fewer external libraries** to track for security patches.
- **Better performance** under load, thanks to HTTP/2 + virtual threads.

---

## Summary Table — What the Client Gets

| Feature                          | Business value                         | One-line summary                           |
|----------------------------------|----------------------------------------|--------------------------------------------|
| <b>Virtual Threads</b>           | Lower hardware cost, higher throughput | Same server handles 10–100× more customers |
| <b>Records</b>                   | Faster development, fewer bugs         | 60 lines become 1                          |
| <b>Sealed + Pattern Matching</b> | Safer changes                          | Compiler catches missing cases             |
| <b>Structured Concurrency</b>    | Cleaner parallel work, no leaks        | Parallel tasks live and die together       |
| <b>Text Blocks</b>               | Cleaner customer-facing text           | What you write is what the customer sees   |
| <b>Modern HttpClient</b>         | Faster, simpler integrations           | Built-in, no extra libraries               |

---

## Recommended Order for the Demo

For a client demo we suggest:

1. **Records** — easy to understand, instant visual impact (“60 lines → 1 line”).
2. **Virtual Threads** — strongest business argument, can be shown with a load test.
3. **Sealed + Pattern Matching** — quality and safety message; show how the compiler refuses to build when a case is missing.
4. **Text Blocks + HttpClient** — quick “and there is more” wrap-up.

This order goes from **easy to grasp** → **impressive performance** → **better quality** → **everyday convenience**, which leaves the client with a positive overall impression.